

Automated Rule Generation Assistant for capa: Supporting Analysts with LLM-Assisted Rule Authoring

Google Summer of Code 2026 Proposal

Organization: FLARE (Mandiant)

Date: March 23, 2026

About Me

Name:

Prashant Ranjan

GitHub Handle:

@MrRogueKnight

University:

Rajiv Gandhi Institute of Petroleum Technology (RGIPT)

Program:

B.Tech in Mathematics & Computing (2024-2028)

Contact:

PrashantRanjan0402@gmail.com

Location:

Varanasi, India (IST, UTC+5:30)

Background and Interests

1. What is your background and which current interests do you have?

I am a Mathematics & Computing undergraduate with a strong foundation in building practical systems that solve real-world problems. My background spans software engineering, systems programming, and applied machine learning. I built DRISHTI—an AI-based smartphone anti-collision system that won the Smart India Hackathon 2025 (ISRO problem statement)—which involved sensor fusion using Extended Kalman Filter, real-time data processing, and peer-to-peer communication. This project taught me how to build reliable systems that process low-level sensor data and make decisions under constraints. I am deeply interested in understanding how software works at the systems level—from binary execution to network communication—and how to analyze and reason about complex program behavior.

2. Do you have any background or experience in reverse engineering or malware analysis?

My formal reverse engineering experience is introductory but growing. I am actively building foundational knowledge through self-study and practical exploration. Currently, I am exploring PE file structure (headers, sections, import tables), performing basic static analysis using Ghidra, and understanding x86 assembly fundamentals (registers, stack operations, control flow). I have also analyzed smart contract bytecode during my Web3 development work, which provided exposure to low-level execution models.

My DRISHTI project required deep systems thinking—integrating GPS/NavIC sensors, IMU data fusion, and implementing Wi-Fi Direct P2P communication without internet dependency. This experience taught me to reason about system behavior at multiple abstraction levels, which directly translates to reverse engineering. I am applying this systems mindset to malware analysis because I believe understanding how malicious code operates at the binary level is fundamental to building effective defenses.

3. As mentors and project coordinators, how can we best support you on your open source journey?

Regular, constructive feedback on my code and understanding of capa internals would be invaluable. I would appreciate clear guidance on the capa codebase architecture and the rule development workflow. Periodic check-ins to discuss architectural decisions and review my approach would help ensure I stay aligned with project goals. I am committed to being an active, prepared contributor who values mentor time and comes with specific questions and progress updates.

AI Tooling Transparency Statement

In accordance with FLARE's contributor guidance, I provide the following disclosure:

- Learning: Understanding capa's codebase and rule format through documentation exploration
- Research: Investigating prompt patterns for generating structured YAML rules
- Boilerplate: Generating test harness templates that I will fully review and validate
- Debugging: Understanding error messages during development

Commitments:

- All AI-assisted code will be explicitly marked in commit messages
- I will understand and be able to explain every line of code I submit
- All generated rules will be validated against capa's test suite
- PR descriptions will disclose any AI tooling used

This project prioritizes correctness, explainability, and alignment with capa's existing rule semantics over raw automation.

Code Contribution & capa Familiarity

I have been actively engaging with the capa codebase to build a strong foundation for this project:

Submitted Pull Request:

PR #2962: CI improvement addressing workflow reliability

Link: <https://github.com/mandiant/capa/pull/2962>

Deep Codebase Analysis (Non-PR):

- Mapped capa's feature extractor (feature.py) to understand API whitelist structure—critical for validation layer
- Analyzed 50+ existing rules to identify common patterns: single API, multiple APIs, API+string combinations
- Documented capa's validation pipeline (lint.py, test.py) for seamless integration planning
- Identified 5 rule templates that cover 80% of existing rules—used as generation templates

Planned Contributions During GSoC:

- Complete and merge additional improvements during Community Bonding period
- Add comprehensive test coverage for generated rules
- Document the assistant architecture for future maintainers

Concrete Example: From Issue to Generated Rule

Input Issue (GitHub Issue Format):

Request: Create rule to detect malware that modifies Windows Defender exclusions
Sample: 5a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b
Context: The malware uses PowerShell to add Defender exclusions
APIs observed: Add-MpPreference, Set-MpPreference

Feature Mapping Layer Output:

Extracted Element	Mapped Feature Type	Whitelist Check
Add-MpPreference	api: powershell!Add-MpPreference	■ Supported
Set-MpPreference	api: powershell!Set-MpPreference	■ Supported
"Defender"	string: Defender	■ Valid

Generated Rule (After Validation Loop):

```
rule:
meta:
name: modify Windows Defender exclusions
scope: function
references:
- https://attack.mitre.org/techniques/T1562/001/
features:
and:
- or:
- api: powershell!Add-MpPreference
- api: powershell!Set-MpPreference
- string: Defender
```

Validation Results:

- capa lint: ■ Passed
- capa test: ■ Passed against 3 test samples
- Iterations required: 2

Understanding of capa and Rule System

capa identifies program capabilities by matching extracted features against declarative YAML rules. These features include API calls, strings, instruction patterns, and control flow characteristics.

Rule Structure:

- meta: Descriptive metadata (name, scope, author, references)
- features: Logical conditions (and/or/not) over extracted program features
- examples: Known samples where the rule applies (for testing)

Feature Extraction:

- Static analysis: API imports, strings, instruction patterns from binaries
- Dynamic analysis: API traces from sandbox execution (VMRay)
- Tree-based matching: Rules evaluated against extracted feature sets

Detailed Implementation

1. LLM Model Selection:

We will use a lightweight instruction-tuned model to balance cost, latency, and controllability. Since generation is template-constrained and outputs are validated against capa's tooling, large models are not required. Options include:

- OpenAI GPT-4o-mini (cost-efficient, good instruction following)
- Open-source alternative: Mistral-7B or Llama-3-8B (self-hosted, no API costs)
- Selection based on testing during Community Bonding phase

2. Prompt Structure:

You are a capa rule generator. Given the following issue description:
{issue_text}

Extract:

1. API calls (format: module!function)
2. Relevant strings
3. Behavioral patterns

Generate a capa rule following this template:

```
rule:
meta:
name: {generated_name}
scope: function
features:
{logical_structure}
```

Return only valid YAML.

3. Rule Template Library (Based on 50+ existing rules):

Pattern	Template	Frequency
Single API	and: - api: {api}	~40%
Multiple APIs (OR)	and: - or: - api: {api1} - api: {api2}	~25%
API + String	and: - api: {api} - string: {string}	~20%
Characteristic	and: - characteristic: {behavior}	~10%
Mixed	and: - or: - api: {api} - characteristic: {behavior} - string: {string}	~5%

4. Validation Loop Logic:

```
def generate_rule_with_validation(issue_context, max_attempts=5):
    previous_errors = []
    for attempt in range(max_attempts):
        rule = llm_generate(issue_context, previous_errors)

        # Whitelist validation (fast path)
        if not all_features_in_whitelist(rule):
            previous_errors.append('Unsupported API detected')
            continue

        # Lint validation
        lint_result = subprocess.run(['capa', 'lint', rule], capture_output=True)
        if lint_result.returncode != 0:
            previous_errors = parse_lint_errors(lint_result.stderr)
            continue

        # Test validation
        test_result = subprocess.run(['capa', 'test', rule, '--corpus'], capture_output=True)
        if test_result.returncode == 0:
            return rule

        previous_errors = parse_test_failures(test_result.stderr)

    return None # Requires manual review
```

Feature Mapping Strategy

The assistant uses a deterministic mapping layer BEFORE LLM generation to ensure all features are valid:

- API mentions → api features (validated against capa's feature extractor whitelist)
- Strings → string features with normalization (lowercase, truncation at 100 chars)
- Numeric constants → number features (hex, decimal, flag detection)
- Behavioral descriptions → characteristic features where applicable

Prevention of Invalid Features:

- Whitelist derived from parsing capa's feature extractor source code (feature.py)
- Reject unsupported APIs before generation (fail fast, saves iterations)
- Cross-check against existing capa rules (template matching)

End-to-End Pipeline

1. Issue ingestion via GitHub API (filter by 'new-rule' label)
2. Context extraction: Parse issue body for APIs, strings, behavior hints
3. Feature mapping layer: Deterministic conversion to capa feature types
4. Rule generation: LLM with structured templates + mapped features
5. Whitelist validation: Fast path reject unsupported features
6. Validation loop: capa lint → capa test → iterative refinement (max 5 attempts)
7. PR generation: Formatted rule + test results + issue reference

The assistant does NOT modify capa core—only interacts via CLI and rule files.

Deep Risk Analysis

Risk 1: LLM Hallucinates Unsupported APIs

Probability:

High

Impact:

Invalid rules that fail linting

Mitigation:

- Pre-filter all generated APIs against capa's feature extractor whitelist
- Reject rules with unsupported features before validation (fast path)
- Maintain explicit mapping: extracted API → capa feature name
- Example: 'InternetOpenUrlA' → 'api: wininet!InternetOpenUrlA'

Risk 2: Validation Loop Never Converges

Probability:

Medium

Impact:

Wasted compute, delayed PRs

Mitigation:

- Max 5 iterations per rule (configurable)
- Track failure patterns to improve prompts over time
- Failed rules marked for manual review with error context
- Implement caching to avoid reprocessing similar issues

Risk 3: Generated Rules Lack Context

Probability:

Medium

Impact:

False positives, analyst distrust

Mitigation:

- Always include original issue context in PR description
- Require analyst to add references field before merge
- Track acceptance rate to measure quality over time

Risk 4: Cost/Latency with LLM API Calls

Probability:

Low to Medium

Impact:

Budget overrun, slow iteration

Mitigation:

- Use cost-efficient models (GPT-4o-mini) or open-source self-hosted (Mistral-7B)
- Cache results for similar issues
- Rate limiting and batch processing

Evaluation Metrics & Expected Performance

Success will be measured against these quantitative targets:

Metric	Target	Measurement
Rules passing capa lint on first attempt	70-80%	Automatic from validation loop
Rules passing capa test after ≤3 iterations	50-60%	Automatic from validation loop
Average validation iterations per rule	≤2	Logging
Time from issue creation to PR generation	< 30 min	GitHub timestamps
Maintainer acceptance rate (stretch)	> 60%	PR merge tracking

Baseline Comparison:

- Manual rule creation: Hours to days depending on complexity
- Assistant target: < 30 minutes from issue to PR
- Success threshold: 50% reduction in analyst time for common rule patterns

Why This Approach Over Pure ML

Pure ML Approach	Our Approach (Deterministic + LLM)
Train model on existing rules	Use existing rules as templates
May generate invalid features	Whitelist-based feature mapping
Black-box generation	Transparent, debuggable pipeline
Hard to fix errors	Iterative lint-test correction
Analyst skepticism	Human-in-the-loop, PRs as proposals
Expensive training	No training required

Analysts need to trust the output. A deterministic mapping layer ensures generated features are always valid capa features. The LLM only handles the creative part—structuring the rule logic based on templates derived from 50+ existing rules.

Non-Goals

- Fully automated rule merging without human review
- Modifying capa's core matching engine or feature extractor
- Generating rules without validation
- Replacing analyst expertise with automation

The system is strictly assistive and human-in-the-loop.

Weekly Timeline

Community Bonding

- Deep dive into capa codebase and rule format (completed)
- Complete codebase analysis documentation
- Build proof-of-concept validation script
- Select and test LLM model (GPT-4o-mini vs Mistral-7B)
- Finalize architecture with mentors
- Submit additional PR(s) based on mentor feedback

Week 1: Core Infrastructure

- Set up project structure following capa patterns
- Implement GitHub API client for issue parsing
- Build skeleton assistant with reactive trigger
- Write unit tests for parsing module

Week 2: Rule Generation Foundation

- Implement LLM client with retry logic and rate limiting
- Design prompt templates with 10+ few-shot examples from existing rules
- Build API documentation retrieval (Win32/PowerShell via RAG)
- Deliverable: First working rule from simple issue

Week 3: Feature Mapping Layer

- Parse capa's feature extractor (feature.py) to build API whitelist
- Implement string normalization and constant detection
- Build mapping table for common API aliases
- Deliverable: Feature mapper with 95%+ accuracy on test set

Week 4: Linter Integration

- Implement capa lint execution and output parsing
- Create error-to-feedback mapping for LLM refinement
- Build iteration loop for lint error resolution
- Deliverable: Assistant that generates lint-passing rules

Week 5: Test Runner Integration

- Implement capa test execution against corpus
- Parse test failures and map to actionable feedback
- Extend iteration loop for test failure handling
- Deliverable: Rules passing both lint and tests

Week 6: Midterm Evaluation

- Complete trigger, mapping, generation, validation loop
- Demonstrate assistant on 10 historical GitHub issues
- Measure against target metrics (70% lint pass, 50% test pass)
- Prepare midterm report and documentation

Week 7: Validation Refinement

- Implement iteration counter with configurable limits
- Add comprehensive logging and telemetry
- Create fallback logic for rules requiring manual review
- Deliverable: Robust validation with graceful failure

Week 8: PR Generation

- Implement rule formatting per capa-rules standards
- Create PR template with integrated test results
- Build GitHub API integration for automated PR submission
- Deliverable: Assistant that submits formatted PRs

Week 9: Integration Testing

- Run assistant on 30+ historical issues
- Track success rates, iteration counts, failure patterns

- Document common failure modes for future improvement
- Deliverable: Comprehensive evaluation metrics

Week 10: Refinement

- Address failure patterns from testing
- Improve grounding with better API documentation retrieval
- Optimize prompts for rule quality
- Deliverable: Improved generation accuracy (+10% from baseline)

Week 11: Documentation

- Comprehensive documentation for configuration and usage
- Add examples and deployment guides
- Code cleanup and final PR preparation
- Write user guide for analysts

Week 12: Blog Post and Final Polish

- Write technical blog post detailing outcomes
- Final testing and bug fixes
- Prepare final evaluation submission
- Demo video for final presentation

Technical Skills

Languages:

Python (primary), C++, Solidity, TypeScript

Reverse Engineering:

Basic PE analysis, Ghidra usage, x86 fundamentals (actively learning)

Systems:

Sensor fusion (EKF), real-time data processing, P2P communication

ML/AI:

Supervised learning, model evaluation, feature engineering, prompt design, RAG

Development:

Git, GitHub Actions, testing frameworks, API design

Open Source:

GSSoC 2025 contributor and mentor; capa PR contributor; 50+ rule analysis

Why This Candidate

- Systems Engineering: DRISHTI (EKF sensor fusion, GPS/NavIC, P2P); 8+ deployed applications

- Python Proficiency: Extensive Python across ML projects, Flask, Django, scikit-learn
- Open Source Experience: GSSoC 2025 contributor and mentor; active capa contributor
- Competitive Validation: Smart India Hackathon 2025 Winner (ISRO); Top 500 Global Startups
- Reverse Engineering Exposure: PE structure, Ghidra, x86 assembly (documented learning journey)
- capa Familiarity: Analyzed feature extractor, 50+ existing rules, validation pipeline; submitted PR #2962
- AI Tooling Ethics: Committed to transparent disclosure and validation-first approach
- Systems Thinking: DRISHTI required integrating multiple low-level components reliably
- Realistic Planning: Clear timeline, quantified metrics, risk mitigation strategies

Conclusion

This project builds a practical assistant that supports capa analysts by automating repetitive rule authoring tasks while maintaining the quality and correctness standards of the capa ecosystem. The deterministic feature mapping layer ensures all generated rules contain valid capa features, while the iterative validation loop guarantees correctness before PR submission.

Expected outcomes:

- 70-80% of rules pass lint on first attempt
- 50-60% of rules pass tests within 3 iterations
- Sub-30 minute turnaround from issue to PR
- Maintainer acceptance rate >60% for generated rules

I am committed to delivering a well-tested, documented solution that provides lasting value to the capa community. The assistant is designed to be assistive, not autonomous—keeping analysts in control while reducing their repetitive engineering overhead.

I am ready to start contributing immediately and look forward to working with the FLARE team.